3일차 수업 메모장

#변수

개발자가 지정해주는 값이랑 변수 간의 관계로서 만들어지는 값을 구분해야함

ex) 사과 1000원짜리 10개의 값 계산에서 총 값은 변수 간의 관계로 만들어지는 값임

장바구니의 총비용을 구할 때 ABC의 단가(변수)를 지정하고

갯수를 알아내서 단가*개수를 해서 총비용을 계산하면 되지만

갯수라는 변수 없이 물건을 하나하나 꺼내서 더하면 된다.

→ 두가지의 차이를 문장으로 말할 수 있어야함.

내가 이해한 거 : a,b,c의 단가 변수를 지정해주고 장바구니라는 시퀀스(list)의합계를 구하라고 하면 된다.

교수님 왈 - 곱하기와 더하기가 있으면 더하기를 추천함

컴퓨터는 기본적으로 곱하기 나누기 못함. 더하기 빼기가 기본임.

더하기가 더 빠름.

플로우차트 그리는 연습을 해야함 그리고 플로우 차트에서 코드 변환 연습

- 장바구니 총 비용계산 flowchart

1.단가 대입

2.총비용 초기화 : 0

3.장바구니에 물건이 있는지 확인

4.while true면

물건이 있으면 물건 하나를 빼

총비용 + 물건단가를 더해

while true로 돌아가서 반복

5.while false면

print (총비용)

empty' ' 있으면 T 없으면 F

장바구니 플로우차트

→ 플로우차트 → 소스코드

```
A = int(input("2-9숫자" ))
if 1< A <10:
    B = 1
    while True:
        if B<10:
            C=A*B
            print(A,"*",B,"=",C)
            B +=1
        else:
            break
print("끝")
```

→코드 단순화 해보기

```
A = int(input("2-9숫자: "))

if 1 < A < 10:
    for B in range(1, 10):  # B를 1부터 9까지 자동으로 반복!
        print(A, "*", B, "=", A * B)

print("끝")
```

→*를 더하기 코드로 바꿔보기

```
A = int(input("2-9숫자: "))

if 1 < A < 10:
    for B in range(1, 10):
```

```
        C = 0
        for _ in range(B):
            C += A

        print(A, "*", B, "=", C)

print("끝")
```

질문 : 컴퓨터는 원래 덧셈 뺄샘만 할 줄 알아서 덧셈이 속도가 더 빠르기 때문에 곱셈 코드를 더하기 코드로 바꾸는 노력을 한다고 했는데 왜 이 두 코드 중에 더하기 코드가 루프 횟수가 늘어서 더 느리다고 하는지?

답 : 교수님 말대로 실제로는 그렇지 않고 덧셈이 더 빠름

#운영체제(OS)

스크린샷 2026-03-05 143031.png

가상머신이 어떻게 동작하는지도 알면 좋음

렉시컬 분석을 통해 토큰 분리 →파싱 →트리구조→컴파일러를 통해 바이트 코드로 변환 →PVM이 바이트코드를 하나씩 읽어서 하드웨어로 보내는데 스택과 힙 관련해서 찾아봐야 함.

→

1. 스택 (Stack): "작업 접시 쌓기"
스택은 함수가 호출될 때마다 생성되는 **임시 작업 공간**입니다.
- **특징**: LIFO(Last-In, First-Out, 후입선출) 구조입니다. 나중에 들어온 게 먼저 나갑니다.
- **저장되는 것**:
  - **지역 변수**: 함수 안에서 잠깐 쓰고 버릴 변수들.
  - **함수 호출 정보**: "이 함수가 끝나면 어디로 돌아가야 하지?" 같은 정보.
- **PVM의 역할**: 바이트코드를 읽으면서 연산을 할 때, 숫자를 스택에 'Push'하고 계산 결과를 'Pop'하며 아주 빠르게 처리합니다.

2. 힙 (Heap): "커다란 창고"
힙은 파이썬의 모든 **객체(Object)**가 실제로 저장되는 **거대한 공유 메모리 공간**입니다.

- **특징**: 크기가 크고 자유롭지만, 관리가 복잡합니다.
- **저장되는 것**:
  - 우리가 만든 숫자, 문자열, 리스트, 클래스 인스턴스 등 **모든 실체**.
- **파이썬의 독특한 점**:
  - `A = 2`라고 하면, 숫자 `2`라는 객체는 **힙**에 생성됩니다.
  - **스택**에 있는 변수 `A`는 힙에 있는 `2`의 **주소값(메모리 위치)**만 가리키고 있습니다.

3. 왜 힙과 스택을 같이 쓸까요? (중요!)

이 구조를 알면 파이썬의 성능 비밀이 풀립니다.

구분	스택 (Stack)	힙 (Heap)
속도	매우 빠름 (자동 관리)	상대적 느림 (찾는 과정 필요)
수명	함수 종료 시 즉시 삭제	누군가 참조하고 있으면 유지
관리 주체	CPU/PVM이 자동으로 처리	**가비지 컬렉터(GC)**가 처리

💡 **핵심 요약**

파이썬에서 계산을 할 때는 **스택**에서 바쁘게 움직이고, 그 계산 결과로 만들어진 실제 데이터 덩어리들은 **힙**이라는 넓은 창고에 보관됩니다.

4. 가비지 컬렉터 (Garbage Collector)

공부하시면서 **힙** 관련해서 꼭 찾아보셔야 할 게 바로 이겁니다. 힙은 창고가 넓어서 쓰레기가 쌓이기 쉽거든요. 파이썬은 **Reference Counting(참조 횟수 계산)** 방식을 써서, 아무도 쳐다보지 않는(참조하지 않는) 힙 공간의 객체를 자동으로 치워줍니다.

할일 : 파이참도 다뤄보기, vscode랑 colab도 숙달필요

구글 드라이브에 환경만들기